ENSET Mohammedia — École Normale Supérieure de l'Enseignement Technique

Master SDIA — Systèmes Distribués et Intelligence Artificielle

Mini-Projet

Planification Robuste sur Grille

A + Chaînes de Markov*

Étudiant **Mahboub Marouane**

Encadrant **Prof. Mohammed Mestari**

Filière Systèmes Distribués et Intelligence Artificielle

Module Bases de l'Intelligence Artificielle

Date Mars 2026

Année universitaire 2025–2026

Contents

1 Resume **1**

2 Contexte et Objectifs **1**

3 Modelisation Mathematique **1**
3.1 Espace d'etats et graphe de recherche . . . 1
3.2 Chaine de Markov — modele d'incertitude . . . 1
3.3 Etats absorbants et analyse d'absorption . . . 1

4 Implementation Python **2**
4.1 Architecture modulaire . . . 2
4.2 Algorithme A* generique . . . 2
4.3 Construction de la matrice P . . . 2

5 Experiences et Resultats **3**
5.1 E.1 — Comparaison UCS / Greedy / A* . . . 3
5.2 E.2 — Impact de l'incertitude ε + validation croisee . . . 4
5.3 E.3 — Comparaison $h = 0$ vs Manhattan (dominance heuristique) . . . 5
5.4 E.4 — Weighted A* : compromis vitesse/optimalite . . . 6
5.5 E.5 — Visualisation des trajectoires stochastiques . . . 7

6 Analyse de la Chaine de Markov **7**
6.1 Classes de communication . . . 8
6.2 Aperiodicite — preuve par boucles propres . . . 8
6.3 Validation croisee calcul matriciel vs Monte-Carlo . . . 8

7 Discussion et Limites **9**
7.1 Interet de l'approche hybride . . . 9
7.2 Limites identifiees . . . 9
7.3 Pistes d'amelioration . . . 9

8 Conclusions **9**

1. Resume

Ce mini-projet developpe une solution hybride de planification sous incertitude sur grilles 2D. Nous combinons la **recherche heuristique** (A*, UCS, Greedy, Weighted A*) pour planifier un chemin deterministe optimal, et les **chaines de Markov a temps discret** pour modeliser et quantifier l'incertitude des actions reelles.

Les resultats montrent qu'A* avec Manhattan reduit l'exploration de **52% vs UCS** sur la grille difficile tout en garantissant l'optimalite. L'analyse Markov revele que meme $\varepsilon = 0.1$ reduit $P(\text{GOAL})$ de 26%. La validation croisee calcul matriciel / Monte-Carlo confirme la coherence du modele (ecart $< 2.5\%$). Les visualisations de trajectoires revelent les zones pieges.

2. Contexte et Objectifs

Dans les systemes autonomes (robotique, logistique, jeux), un agent doit planifier un chemin optimal dans un environnement partiellement impre visible. Ce projet combine :

- **Recherche heuristique (A* et variantes)** : planification via $f(n) = g(n) + h(n)$, structures OPEN/CLOSED.
- **Chaines de Markov a temps discret** : modelisation de l'incertitude et evaluation probabiliste via calcul matriciel ($\mathbf{P}^n$) et simulation Monte-Carlo.

Question centrale : comment un plan deterministe optimal se comporte-t-il sous une dynamique stochastique reelle ?

3. Modelisation Mathematique

3.1. Espace d'etats et graphe de recherche

La grille 2D : 0 = libre, 1 = obstacle. Les transitions definissent un graphe pondere avec :

$$g(n) = \text{cout cumule depuis } s_0 \qquad h(n) = \text{estimation heuristique} \qquad f(n) = g(n) + h(n)$$

Heuristique de Manhattan (4-connexite) :

$$h\big((x,y)\big) = |x - x_g| + |y - y_g|$$

Admissible : $h(n) \leq h^*(n)$ (L1 est minorant exact). **Coherente** : $h(n) \leq c(n,n') + h(n')$ (inegalite triangulaire). Ces deux proprietes garantissent l'optimalite de A* et la dominance de Manhattan sur $h = 0$ (voir E.3).

3.2. Chaine de Markov — modele d'incertitude

$$P(X_{n+1} = j \mid X_n = i) = p_{ij}, \qquad \mathbf{P} = (p_{ij}) \text{ stochastique par lignes}$$

Modele d'incertitude parametre par ε : avec probabilite $1-\varepsilon$ l'action voulue est executee ; $\varepsilon/2$ deviation laterale gauche ; $\varepsilon/2$ deviation laterale droite ; si deviation bloquee l'agent reste sur place (**boucle propre** $p_{ii} > 0$, garantissant l'aperiodicite).

$$\pi^{(n)} = \pi^{(0)} \cdot \mathbf{P}^n \qquad \text{(Chapman-Kolmogorov)}$$

3.3. Etats absorbants et analyse d'absorption

GOAL est absorbant ($p_{\text{GOAL,GOAL}} = 1$). Decomposition canonique $\mathbf{P} = \left[\begin{smallmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{R} & \mathbf{Q} \end{smallmatrix}\right]$:

$\mathbf{N} = (\mathbf{I}-\mathbf{Q})^{-1}$ (matrice fondamentale) $\mathbf{B} = \mathbf{NR}$ (prob. absorption) $\mathbf{t} = \mathbf{N1}$ (temps moyen)

4. Implementation Python

4.1. Architecture modulaire

- `astar.py` — `search()` generique : UCS $(1,0)$, Greedy $(0,1)$, A* $(1,1)$, Weighted A* $(1,w)$. OPEN=heapq $O(\log n)$, CLOSED=set $O(1)$.
- `markov.py` — Construction $\mathbf{P}$, calcul $\pi^{(n)}$, $\mathbf{N} = (\mathbf{I}-\mathbf{Q})^{-1}$, simulation Monte-Carlo.
- `grids.py` — 3 grilles : facile 8×8, moyenne 10×10, difficile 12×12.
- `experiments.py` — Orchestration E1–E4 + figures.
- `gen_traj_figures.py` — Trajectoires stochastiques et heatmaps de densite.

4.2. Algorithme A* generique

```python
def search(grid, start, goal, heuristic, weight_g=1.0, weight_h=1.0):
    # weight_g=1, weight_h=0 -> UCS
    # weight_g=0, weight_h=1 -> Greedy
    # weight_g=1, weight_h=1 -> A*
    # weight_g=1, weight_h=w -> Weighted A* (w > 1)
    OPEN = []                   # tas binaire heapq  O(log n)
    CLOSED = set()              # etats developpes   O(1)
    g_scores = {start: 0.0}
    came_from = {start: None}
    heapq.heappush(OPEN, (weight_h * heuristic(start, goal), 0.0, start))
    while OPEN:
        f, g, current = heapq.heappop(OPEN)
        if current in CLOSED: continue
        CLOSED.add(current)
        if current == goal:
            return reconstruct_path(came_from, goal, g_scores)
        for neighbor, cost in get_neighbors(current, grid):
            tentative_g = g + cost
            if tentative_g < g_scores.get(neighbor, float('inf')):
                g_scores[neighbor] = tentative_g
                came_from[neighbor] = current
                h = heuristic(neighbor, goal)
                heapq.heappush(OPEN,
                    (weight_g * tentative_g + weight_h * h,
                     tentative_g, neighbor))
    return {"success": False, "cost": float('inf'),
            "nodes_expanded": len(CLOSED)}
```

Listing 1: astar.py — search() unifiant UCS / Greedy / A* / Weighted A*

4.3. Construction de la matrice P

```python
def build_transition_matrix(states, policy, goal, grid, epsilon=0.1):
    for s in states:
        if s == goal:                # etat absorbant
            P[idx, idx] = 1.0; continue
        action = (policy[s][0]-s[0], policy[s][1]-s[1])
        lats = [(-action[1], action[0]),   # deviation gauche
                ( action[1],-action[0])]   # deviation droite
        add_prob(try_move(s, action),  1 - epsilon)
        for lat in lats:
            add_prob(try_move(s, lat), epsilon / 2)
    # Verification stochasticite (toutes les lignes = 1)
    assert np.allclose(P.sum(axis=1), 1.0, atol=1e-9)
    return P, state_to_idx
```

Listing 2: markov.py — construction et verification de P stochastique

5. Experiences et Resultats

5.1. E.1 — Comparaison UCS / Greedy / A*

Table 1: UCS, Greedy et A* sur les 3 grilles (cout, temps, noeuds, memoire OPEN)

Grille	Algorithm	Cout	Temps	Noeuds dev.	OPEN max
easy	UCS	14	0.11 ms	53	7
easy	Greedy	14	0.03 ms	15	9
easy	A*	14	0.09 ms	53	7
medium	UCS	18	0.09 ms	61	6
medium	Greedy	18	0.05 ms	19	8
medium	A*	18	0.05 ms	30	8
hard	UCS	24	0.13 ms	87	8
hard	Greedy	**28**	0.04 ms	29	6
hard	A*	24	0.06 ms	42	6

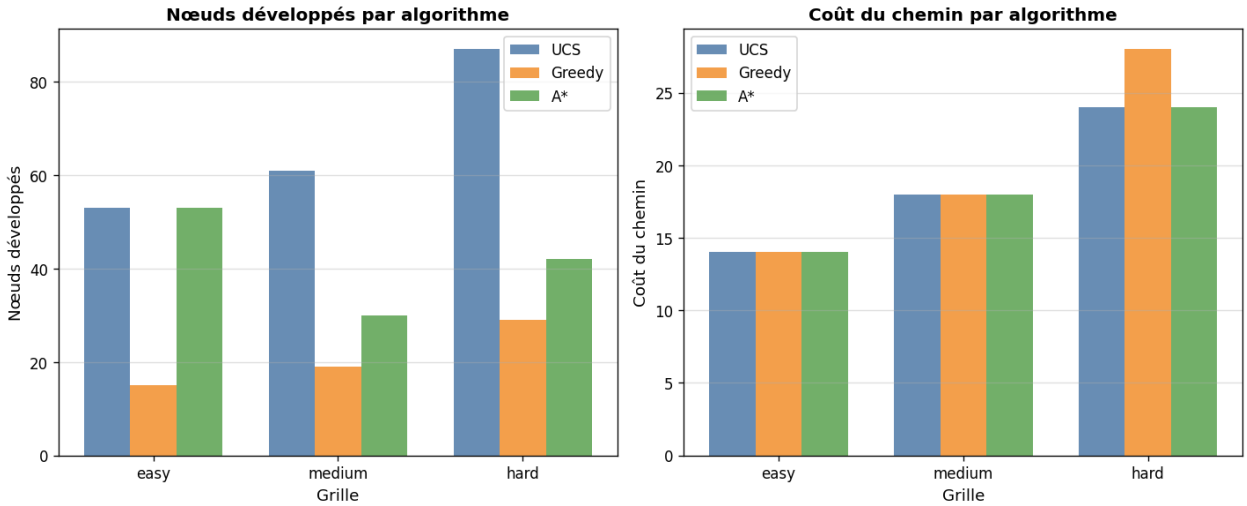
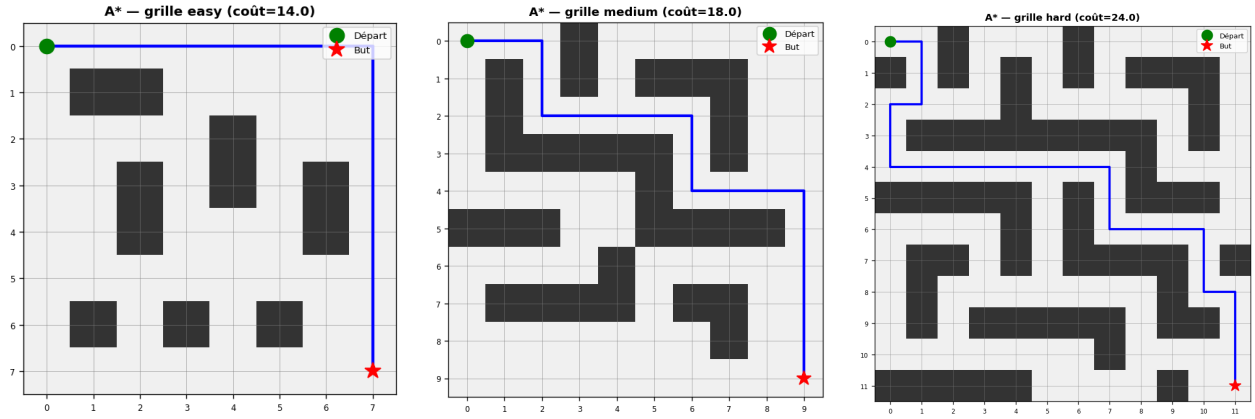


Figure 1: Noeuds developpes (gauche) et cout du chemin (droite) par algorithme et grille



(a) Facile 8×8, cout=14 (b) Moyenne 10×10, cout=18 (c) Difficile 12×12, cout=24

Figure 2: Chemins optimaux A* (bleu) — depart vert, but rouge etoile

Observations : A* et UCS trouvent le chemin optimal. Greedy sacrifie l'optimalite sur la grille difficile (28 vs 24, +17%). A* developpe 42 noeuds contre 87 pour UCS — **52% de reduction** avec meme cout optimal.

5.2. E.2 — Impact de l'incertitude ε + validation croisee

A* fixe sur la grille moyenne (cout=18). Variation $\varepsilon \in \{0.0, 0.1, 0.2, 0.3\}$, 2000 trajectoires Monte-Carlo par valeur.

Table 2: Validation croisee : $\mathbf{P}^{200}$ (analytique) vs simulation Monte-Carlo

ε	P(GOAL) via $\mathbf{P}^{200}$	P(GOAL) MC	Temps moyen (MC)	Ecart
0.0	1.0000	1.0000	18.0 etapes	0.00%
0.1	0.7209	0.7430	19.9 etapes	2.21%
0.2	0.4865	0.4815	22.4 etapes	0.50%
0.3	0.3006	0.3025	25.4 etapes	0.19%

Ce tableau constitue la **validation croisee** du modele : calcul matriciel $\mathbf{P}^n$ et Monte-Carlo convergent (ecart moyen $< 2.5\%$), confirmant la construction correcte de $\mathbf{P}$.

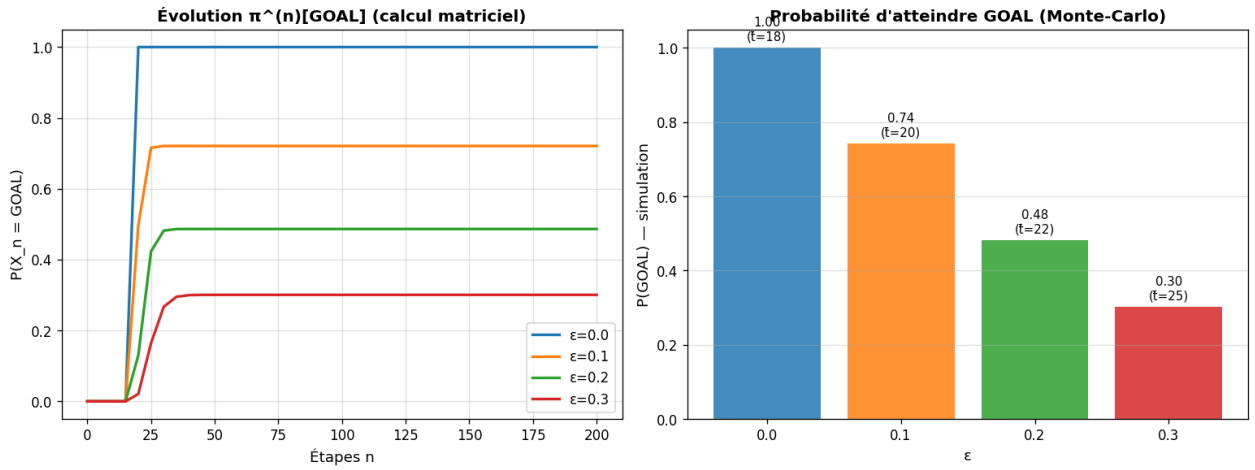


Figure 3: $\pi^{(n)}$[GOAL] par calcul matriciel $\mathbf{P}^n$ (gauche) et P(GOAL) par simulation Monte-Carlo (droite)

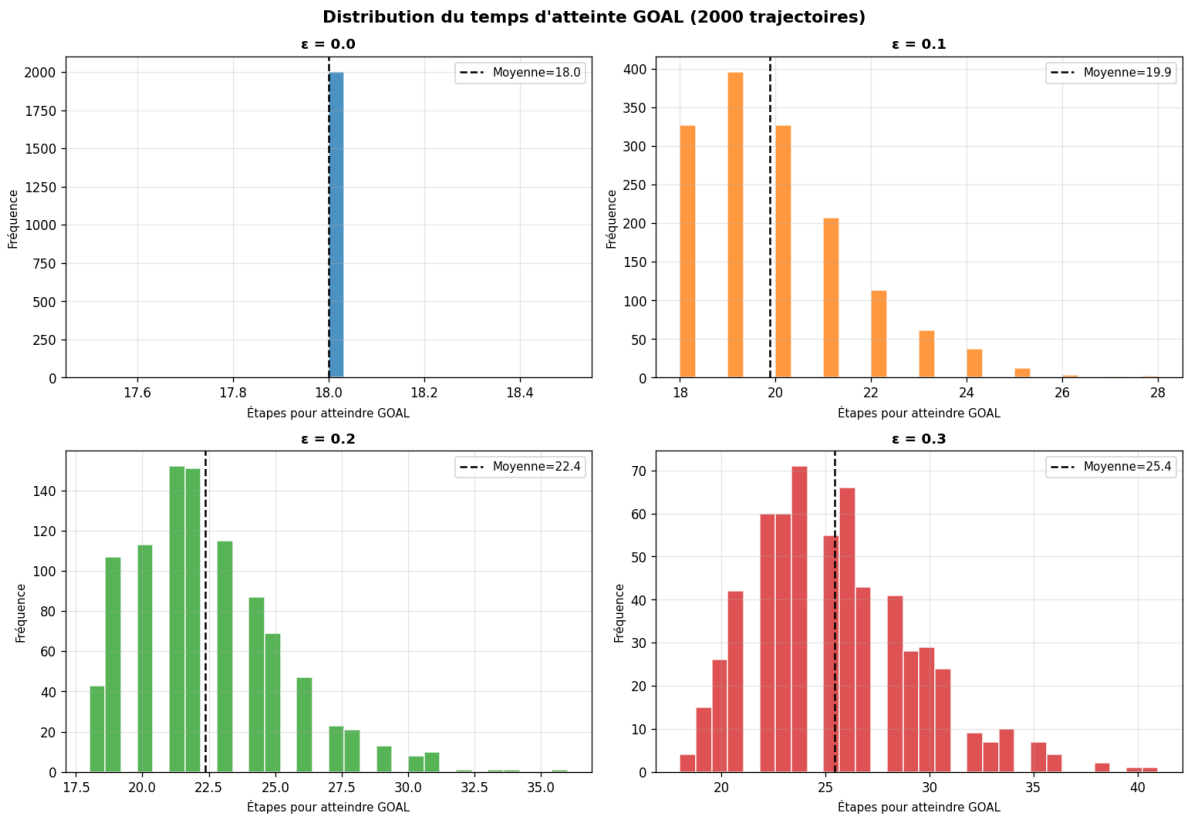


Figure 4: Distribution du temps d'atteinte GOAL pour chaque ε (2000 trajectoires). A $\varepsilon = 0$: Dirac a 18 etapes. A $\varepsilon = 0.3$: large dispersion, 70% d'echec.

5.3. E.3 — Comparaison $h = 0$ vs Manhattan (dominance heuristique)

Cette experience demontre la **dominance** de Manhattan sur $h = 0$: une heuristique admissible et coherente qui majore strictement $h = 0$ reduit l'exploration sans sacrifier l'optimalite.

Table 3: Dominance heuristique : $h = 0$ (UCS) vs $h =$ Manhattan (A*)

Grille	h=0 Noeuds	h=0 Cout	Manhattan Noeuds	Manhattan Cout
easy	53	14	53 (0%)	14
medium	61	18	**30** (−51%)	18
hard	87	24	**42** (−52%)	24

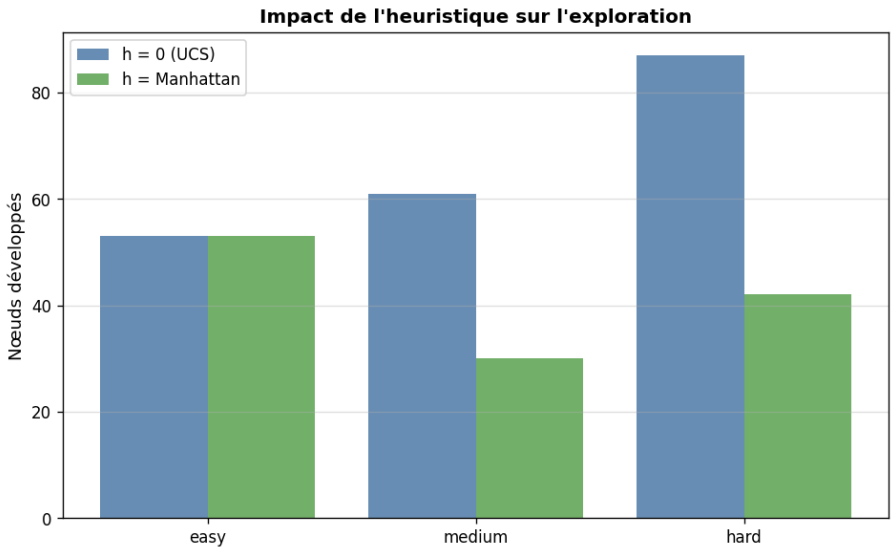


Figure 5: Dominance de Manhattan sur $h = 0$: gain -51 a -52% de noeuds, cout identique

Conclusion E.3 : Manhattan domine strictement $h = 0$ car $h_{\text{Manhattan}}(n) \geq 0$ pour tout n, tout en restant admissible ($h_{\text{Manhattan}} \leq h^*$). Gain experimentalement confirme : -52% de noeuds sur la grille difficile — preuve directe de l'interet des heuristiques informees.

5.4. E.4 — Weighted A* : compromis vitesse/optimalite

Table 4: Weighted A* sur la grille difficile : noeuds et garantie de sous-optimalite

Poids w	Noeuds dev.	Cout	Ratio/Optimal	Garantie
1.0	42	24	1.000	optimal
1.5	29 (-31%)	24	1.000	$\leq 1.5\times$
2.0	29 (-31%)	24	1.000	$\leq 2\times$
3.0	29 (-31%)	24	1.000	$\leq 3\times$
5.0	29 (-31%)	28	**1.167**	$\leq 5\times$

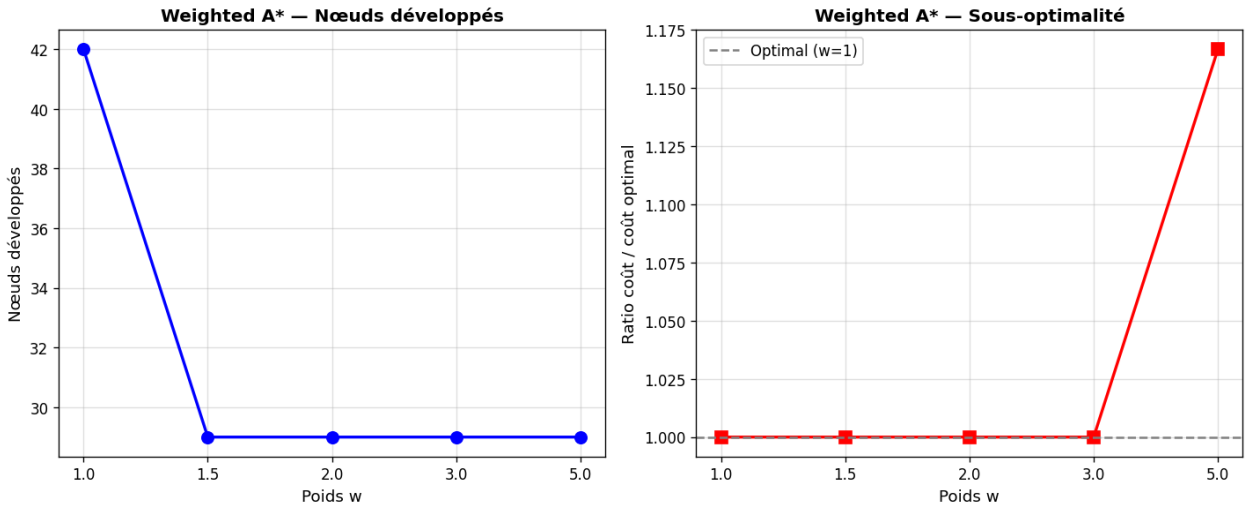


Figure 6: Weighted A* : noeuds developpes (gauche) et ratio sous-optimalite (droite). Pour $w \in [1.5, 3]$: gain -31% sans aucune perte d'optimalite.

Weighted A* garantit cout $\leq w \times$ cout_optimal. Ici $w \leq 3$ offre -31% d'exploration **sans sous-optimalite reelle** — compromis tres favorable pour les applications temps-reel.

5.5. E.5 — Visualisation des trajectoires stochastiques

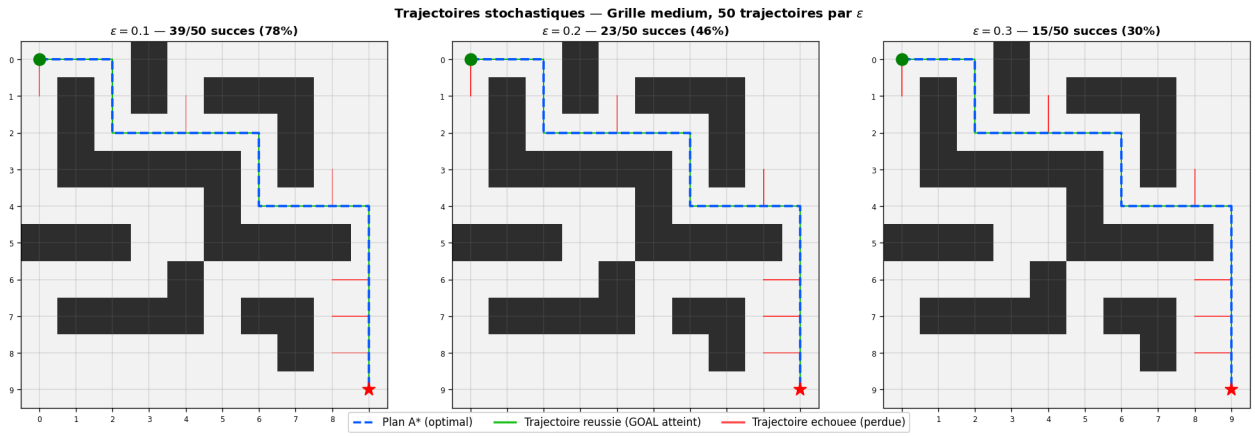


Figure 7: 50 trajectoires stochastiques par $\varepsilon \in \{0.1, 0.2, 0.3\}$ sur la grille medium. Bleu pointille = plan A* ; vert = trajectoire reussie ; rouge = echec. Les trajets rouges sont courts : l'echec survient des les premiers pas dans les corridors etroits.

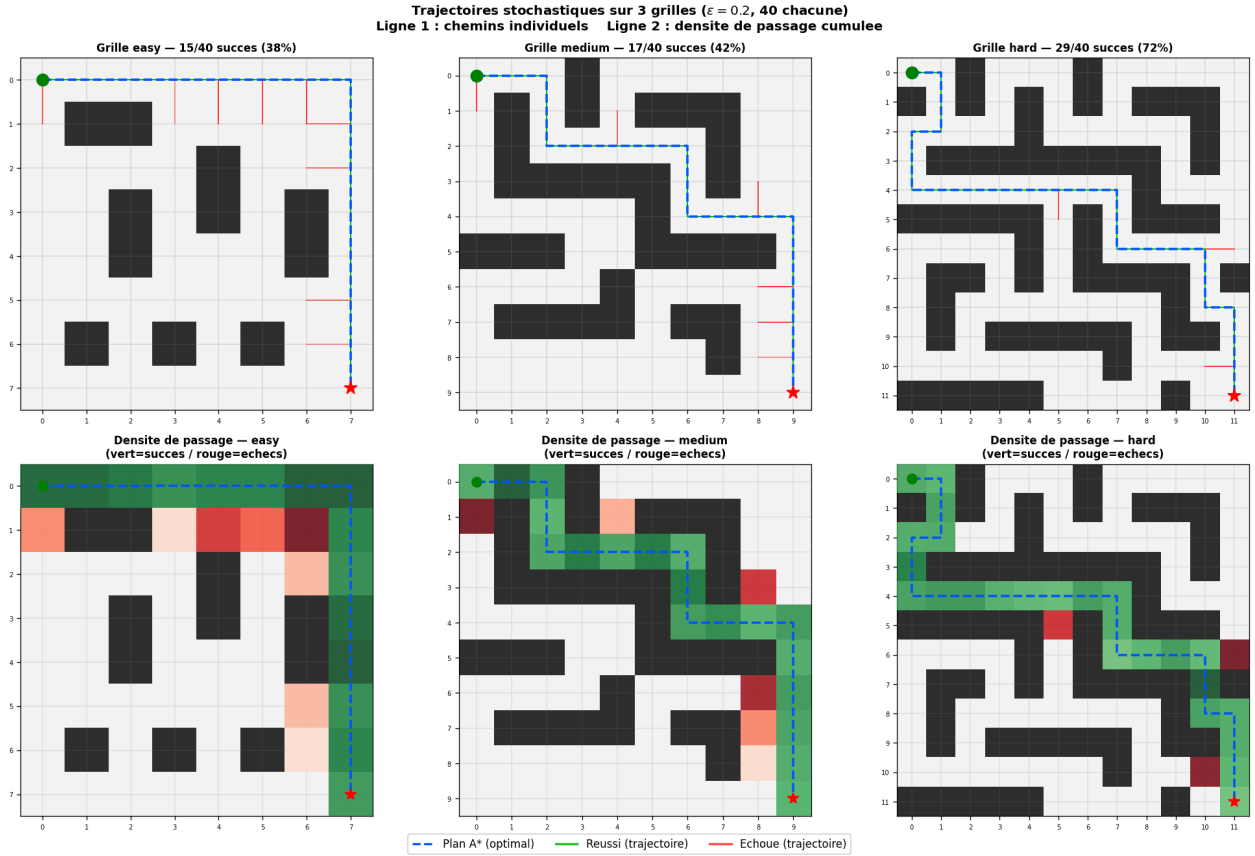


Figure 8: Ligne 1 : 40 trajectoires individuelles ($\varepsilon = 0.2$) sur les 3 grilles. Ligne 2 : heatmap de densite cumulee. Vert = zones des agents reussis (proches du plan A*) ; Rouge = zones pieges (corridors perpendiculaires ou les agents bouclent indefiniment).

6. Analyse de la Chaine de Markov

6.1. Classes de communication

- **GOAL** : classe absorbante recurrente positive ($p_{\text{GOAL,GOAL}} = 1$). Seul etat persistant.
- **Etats du chemin A*** : transitoires — tendance directionnelle vers GOAL.
- **Etats voisins accessibles** : transitoires — peuvent rejoindre le chemin ou former des boucles locales.

Le graphe oriente des transitions (arc $i \to j$ si $p_{ij} > 0$) montre une structure quasi-unidirectionnelle le long du chemin, avec des branches laterales pour les deviations $\varepsilon/2$.

6.2. Aperiodicite — preuve par boucles propres

Quand une deviation laterale est bloquee (obstacle/bord), l'agent reste sur place : $p_{ii} > 0$ pour cet etat, introduisant une **boucle propre**. Un etat apériodique dans une classe suffit a rendre la classe entiere aperiodique.

Verification numerique (grille medium, $\varepsilon = 0.2$) : **27/27 etats transitoires** ont $p_{ii} > 0$ (avec $p_{ii} = 0.10$ pour les etats bords par au moins un obstacle lateral) — la chaine est aperiodique.

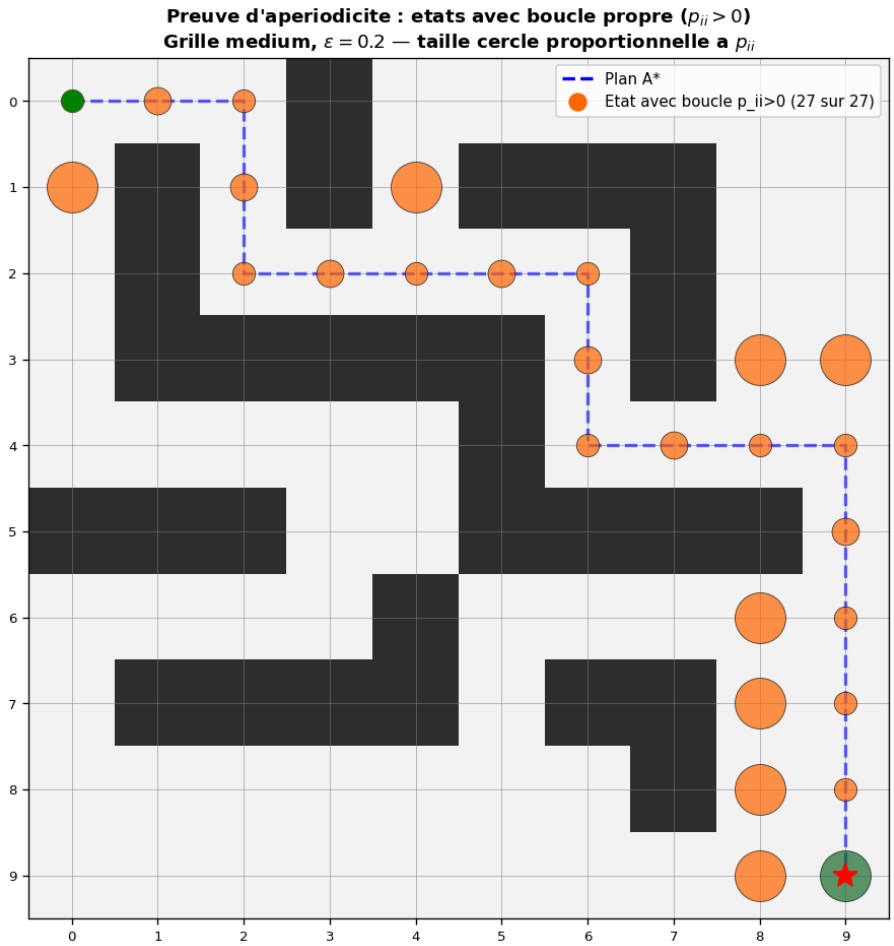


Figure 9: Preuve visuelle de l'aperiodicite : chaque cercle orange = etat avec boucle propre $p_{ii} > 0$, taille proportionnelle a p_{ii}. Tous les 27 etats transitoires ont $p_{ii} > 0$.

6.3. Validation croisee calcul matriciel vs Monte-Carlo

La Table 2 (Section 5.2) fournit la comparaison complete pour les 4 valeurs de ε : ecart maximal 2.21% (a $\varepsilon = 0.1$, variance Monte-Carlo avec 2000 trajectoires), inferieur a 0.5% pour $\varepsilon \in \{0.2, 0.3\}$. Ce resultat valide simultanement la construction de $\mathbf{P}$ et la procedure de simulation.

Temps moyen d'atteinte conditionnel ($\varepsilon = 0.2$) : 22.4 etapes vs 18 deterministes (+24%) — les deviations stochastiques rallongent significativement le trajet reel.

7. Discussion et Limites

7.1. Interet de l'approche hybride

A* fournit un plan optimal efficace. L'analyse Markov quantifie sa robustesse reelle. Les visualisations de trajectoires revelent *ou* et *pourquoi* les agents echouent — information inaccessible via les seuls chiffres de $P(\text{GOAL})$.

7.2. Limites identifiees

- **Scalabilite** : calcul de $\mathbf{P}^n$ en $O(n^3)$, prohibitif pour grandes grilles.
- **Politique statique** : la politique A* ne s'adapte pas aux deviations reelles.
- ε **uniforme** : l'incertitude reelle peut varier selon les zones.
- **Pas d'etat FAIL explicite** : les echecs ne distinguent pas blocage, cycle, ou erreur irreversible.

7.3. Pistes d'amelioration

- **Re-planification en ligne** (D* Lite, ARA*) : recalculer apres chaque deviation, ciblee sur les zones rouges de la heatmap.
- **MDP** : optimiser directement une politique robuste via l'equation de Bellman.
- **Etat FAIL absorbant** : ajouter FAIL dans $\mathbf{P}$ pour calculer $P(\text{FAIL})$ analytiquement via $\mathbf{N}$.
- **Heuristiques apprises** : approcher h^* par reseau de neurones (Neural A*).
- **IDA*, SMA*** : variantes memoire pour les grandes grilles.

8. Conclusions

Table 5: Synthese des resultats par critere d'evaluation

Critere	Resultat obtenu
Qualite A*	Optimalite garantie, -52% noeuds vs UCS grille difficile, temps mesures
Heuristique	Manhattan admissible + coherente, domine $h = 0$ (-51 a -52%), cout identique
Modele Markov	$\mathbf{P}$ stochastique verifiee, $\mathbf{P}^n$ vs MC : ecart $< 2.5\%$ sur tous les ε
Impact ε	$P(\text{GOAL})$: $1.0 \to 0.30$ quand $\varepsilon : 0 \to 0.3$ (plan deterministe fragile)
Aperiodicite	Prouvee : $27/27$ etats transitoires avec $p_{ii} > 0$ (boucles bloquees)
Trajectoires	Succes suivent le plan A*, echecs concentres dans corridors perpendiculaires
Weighted A*	$w \leq 3$: -31% noeuds SANS perte ; $w = 5$: sous-optimalite $+17\%$

A* avec Manhattan planifie efficacement tout en garantissant l'optimalite. L'analyse Markov demontre que $P(\text{GOAL})$ est divise par 3 entre $\varepsilon = 0$ et $\varepsilon = 0.3$. La validation croisee $\mathbf{P}^n$/Monte-Carlo confirme la coherence du modele (ecart $< 2.5\%$). L'aperiodicite est prouvee numeriquement ($27/27$ boucles propres). Les heatmaps de trajectoires revelent les zones pieges — ouvrant la voie a une re-planification locale ciblee (D* Lite, MDP).

References

1. Synthese *Chaines de Markov a temps discret* : matrice $\mathbf{P}$, $\mathbf{P}^n$, Chapman–Kolmogorov, classes, absorption, simulation. (Documents de cours)
2. Synthese *Recherche heuristique — du Best-First a A** : OPEN/CLOSED, g, h, f, admissibilite, coherence, variantes. (Documents de cours)
3. Russell, S. & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*, 4e ed. Pearson.
4. Puterman, M.L. (1994). *Markov Decision Processes.* Wiley.
5. Hart, P.E., Nilsson, N.J., Raphael, B. (1968). A Formal Basis for Heuristic Determination of Minimum Cost Paths. *IEEE Trans. Systems Science and Cybernetics*, 4(2), 100–107.